

MODUL V

API RESTful dan Standar Response

Tujuan

- ◆ Mahasiswa dapat menerapkan versioning pada route API.
- ◆ Mahasiswa dapat membuat BaseController untuk response wrapper konsisten.
- ◆ Mahasiswa dapat menambah sanitasi input di FormRequest.
- ◆ Mahasiswa dapat memperbarui controller agar menggunakan BaseController.
- ◆ Mahasiswa dapat membuat dokumentasi API dasar di [docs/api.md](#) dan menyiapkan Postman Collection.

Materi

1. Desain API RESTful
 - 1.1. Resource-oriented URLs: gunakan kata benda (nouns) dalam bentuk plural, misal [/items](#), [/categories](#).
 - 1.2. Versioning: prefix URL dengan versi, misal [/api/v1/...](#)
 - 1.3. HTTP Status Codes:
 - ✓ 200 OK (GET/PUT success)
 - ✓ 201 Created (POST success)
 - ✓ 204 No Content (DELETE success)
 - ✓ 400 Bad Request (malformed request)
 - ✓ 401 Unauthorized (tidak terautentikasi)
 - ✓ 403 Forbidden (role/otorisasi gagal)
 - ✓ 404 Not Found (resource tidak ada)
 - ✓ 422 Unprocessable Entity (validasi gagal)
 - ✓ 500 Internal Server Error (error server)
2. Standar Response Wrapper & Input Sanitasi
 - 2.1. Semua response JSON dibungkus dengan struktur:
Respon Sukses

```
{
  "success": true,
  "message": "Data berhasil ditampilkan",
  "data": {...}
}
```


Respons Error

```
{
  "success": false,
  "message": "..."
}
```

- 2.2. Input sanitasi di level FormRequest:
 - ✓ `prepareForValidation()` untuk `trim()`, `strip_tags()`, konversi null.
 - ✓ Mencegah injection dan data kacau.
3. Dokumentasi API Sederhana
 - 3.1. Buat file `docs/api.md` atau `README.md` dengan ringkasan setiap endpoint:
Method, URL, headers, body, contoh response.
 - 3.2. Ekspor Postman Collection (.json) sebagai lampiran dokumentasi.

Praktikum

1. Pastikan Anda switch dari branch main ke branch feature/api-design:

```
cd C:\laragon\www\inventory
git checkout main
git pull origin main
git checkout -b feature/api-design
```

2. Tambahkan versioning pada routes

Buka file `routes/api.php`, ubah menjadi:

File: `inventory/routes/api.php`

```
<?php
use Illuminate\Support\Facades\Route;
Route::prefix('v1')->group(function() {
    Route::post('register',
        'App\Http\Controllers\AuthController@register');
    Route::post('login',
        'App\Http\Controllers\AuthController@login');
    Route::middleware('auth:sanctum')->group(function(){
        // Categories
        Route::apiResource('categories',
            'App\Http\Controllers\CategoryController')
            ->except(['destroy']);
        Route::delete('categories/{category}',
            'App\Http\Controllers\CategoryController@destroy')
            ->middleware('role:admin');
        // Items
        Route::apiResource('items',
            'App\Http\Controllers\ItemController')
            ->except(['destroy']);
        Route::delete('items/{item}',
            'App\Http\Controllers\ItemController@destroy')
            ->middleware('role:admin');
    });
});
```

3. Buat BaseController dengan response wrapper
Buat direktori baru `app/Http/Controllers/Api/` dan file `BaseController.php`:

File: inventory/app/Http/Controllers/Api/BaseController.php

```
<?php
namespace App\Http\Controllers\Api;
use App\Http\Controllers\Controller;
class BaseController extends Controller {
    protected function success($data=null, $message=null,
$code=200){
        return response()->json([
            'success' => true,
            'data'     => $data,
            'message' => $message,
        ], $code);
    }
    protected function error($message=null, $code=400){
        return response()->json([
            'success' => false,
            'message' => $message,
        ], $code);
    }
}
```

4. Refactor Controllers agar extend BaseController

4.1 ItemController extend **App\Http\Controllers\Api\BaseController**

File: inventory/app/Http/Controllers/ItemController.php

```
<?php
namespace App\Http\Controllers;
use App\Http\Requests\StoreItemRequest;
use App\Http\Requests\UpdateItemRequest;
use App\Services\ItemService;
use App\Http\Controllers\Api\BaseController;
class ItemController extends BaseController{
    protected ItemService $svc;
    public function __construct(ItemService $svc){
        $this->svc = $svc;
    }
    public function index(){
        return $this->success($this->svc->all());
    }
    public function store(StoreItemRequest $req){
        $item = $this->svc->create($req->validated());
        return $this->success($item, "Item dibuat", 201);
    }
    public function show($id){
        try {
            $item = $this->svc->find($id);
            return $this->success($item);
        } catch (\Exception $e) {
            return $this->error($e->getMessage(), 404);
        }
    }
    public function update(UpdateItemRequest $req, $id){
        $item = $this->svc->update($id, $req->validated());
        return $this->success($item, "Item diperbarui");
    }
}
```

```

        public function destroy($id){
            $this->svc->delete($id);
            return $this->success(null, "Item dihapus", 204);
        }
    }
}

```

4.2. CategoryController → extend BaseController

File: inventory/app/Http/Controllers/CategoryController.php

```

<?php
namespace App\Http\Controllers;
use App\Http\Requests\StoreCategoryRequest;
use App\Http\Requests\UpdateCategoryRequest;
use App\Services\CategoryService;
use App\Http\Controllers\Api\BaseController;
class CategoryController extends BaseController{
    protected CategoryService $svc;
    public function __construct(CategoryService $svc){
        $this->svc = $svc;
    }
    public function index(){
        return $this->success($this->svc->all());
    }
    public function store(StoreCategoryRequest $req){
        $cat = $this->svc->create($req->validated());
        return $this->success($cat, "Kategori dibuat", 201);
    }
    public function show($id){
        try {
            $cat = $this->svc->find($id);
            return $this->success($cat);
        } catch (\Exception $e) {
            return $this->error($e->getMessage(), 404);
        }
    }
    public function update(UpdateCategoryRequest $req, $id){
        $cat = $this->svc->update($id, $req->validated());
        return $this->success($cat, "Kategori diperbarui");
    }
    public function destroy($id){
        $this->svc->delete($id);
        return $this->success(null, "Kategori dihapus", 204);
    }
}

```

5. Tambah sanitasi input di FormRequest

Contoh StoreItemRequest:

File: inventory/app/Http/Requests/StoreItemRequest.php

```

<?php
namespace App\Http\Requests;
use Illuminate\Foundation\Http\FormRequest;
class StoreItemRequest extends FormRequest{
    public function authorize(){
        return true;
    }
}

```

```

    }
    protected function prepareForValidation(){
        $input = $this->all();
        array_walk($input, function (&$val) {
            if (is_string($val)) {
                $val = trim(strip_tags($val));
            }
        });
        $this->merge($input);
    }
    public function rules(){
        return [
            "name" => "required|string|max:255",
            "quantity" => "required|integer|min:0",
            "price" => "required|numeric|min:0",
            "category_id" => "required|exists:categories,id",
        ];
    }
    public function messages(){
        return [
            "name.required" => "Nama item wajib diisi.",
            // ...
        ];
    }
}

```

Lakukan hal sama di UpdateItemRequest, StoreCategoryRequest, UpdateCategoryRequest.

6. Buat dokumentasi API sederhana

6.1. Buat folder dan file:

```

mkdir docs
touch docs/api.md

```

6.2. Isi docs/api.md:

```

File: inventory/docs/api.md
# Inventory System API v1
Base URL: `http://localhost:8000/api/v1`

## Auth
POST /register
  Body: { name, email, password, password_confirmation }
  Response: 201 Created {
    "status":"success",
    "data": { "user":..., "token":... },
    "message":"User registered"
  }
POST /login
  Body: { email, password }
  ...

## Categories
GET    /categories

```

```

POST    /categories { name }
GET      /categories/{id}
PUT      /categories/{id} { name }
DELETE   /categories/{id} (admin only)

## Items
GET      /items
POST     /items { name, quantity, price, category_id }
GET      /items/{id}
PUT      /items/{id}
DELETE   /items/{id} (admin only)

```

6.3. Ekspor Postman Collection

- Buka Postman
- Create Collection "Inventory API v1"
- Tambahkan request sesuai docs
- Export Collection
- Simpan di docs/Inventory.postman_collection.json.

7. Commit & Push

```

git add .
git commit -m "Add API versioning, BaseController, input
sanitization & documentation"
git push -u origin feature/api-design

```

8. Pull Request

8.1. Buka Repository di GitHub

- Pergi ke <https://github.com/USERNAME/inventory>
- Kamu akan melihat notifikasi kuning: "**feature/api-design had recent pushes**" → klik "**Compare & pull request**"

8.2. Isi Form Pull Request

- Base branch: main (branch tujuan merge)
- Compare branch: feature/api-design (branch kamu)
- Title: Isi judul PR
- Setelah form diisi, klik tombol "Create pull request"

8.3. Review Kode (Melihat Perubahan)

- Klik tab "Files changed"
- Lihat perubahan kode:
 - Warna hijau = baris kode yang ditambah
 - Warna merah = baris kode yang dihapus

8.4. Merge Pull Request

- Klik tombol "Merge pull request"
- Klik "Confirm merge"